

FOX_Checklist

What this is: every FoxByte feature and its sub-features — built or not, and how it was tested — so it's clear where we are and what's next. How each feature works is in `FOX_Feature_Implemented.pdf` (same ids).

Last updated: 22 September 2026 · **Reflects:** `main` @ 2778762 (v1.0.1, exact repository path PR #1) + the FoxByte rename and brand indirection (`chore/foxbyte`)

Source: `docs/FOX_Checklist.html` → `make feature-doc` → this PDF

☒ Done & tested — automated tests (unit, VM integration or CI) ☒ Done — tested by hand only  Partial — gaps, or not verified end to end  Known issue
☐ Not started

Sub-features:  built  partly  a known limit  planned, not built ☐ not built yet  broken **The badge rule**

(audit v2 G31, which found 47 rows badged done while carrying an open item): a row is *done* or *done (manual)* only when nothing in it is , ☐ or , and `scripts/test_feature_checklist.sh` fails the build otherwise. A  is a limit of what the feature does and a  →

is a planned extension — neither makes the row's own claim untrue. How thoroughly something was tested belongs in "Tested by", not in the sub-features. · "VM \$n" = section of `make integration` ; "v2 \$n" = `make integration-v2` ; "update" = `make integration-update`

Where we are

Area	Features	Done & tested	Done (manual)	Partial	Known issue	Not started	Progress
A · Install & setup	9	4	3	2	·	·	78%
B · CLI & services	3	3	·	·	·	·	100%
C · Branching & storage	8	7	·	1	·	·	88%
D · Durability	6	5	·	1	·	·	83%
E · Security foundations	3	3	·	·	·	·	100%
F · SQL gateway	7	6	1	·	·	·	100%
G · Accounts & auth	6	5	·	1	·	·	83%
H · Blackbox (record of schema changes)	18	16	·	2	·	·	89%
I · Import & migration	13	5	8	·	·	·	100%
J · ETL pipelines (experimental)	4	2	2	·	·	·	100%
K · APIs, SDKs & agents	6	4	2	·	·	·	100%
L · Web console	11	1	10	·	·	·	100%
M · Build, CI & releases	9	7	1	1	·	·	89%
N · Realtime change feed	1	1	·	·	·	·	100%
Total (features built or started)	104	69	27	8	·	·	92%
Z · Not started (roadmap)	7	1	·	·	·	6	14%

Storage engine — not implemented, by design

FoxByte does **not** have its own storage engine. Data is stored by **PostgreSQL's own engine** (heap tables, B-tree indexes, write-ahead log) running on local disk inside the stock `postgres:18-bookworm` image plus `wal-g` (`postgres:16-bookworm` for an install created before 21 Sep 2026). This was a locked architecture decision: a Neon-style engine that separates compute from storage over the network adds latency to every transaction, and transaction speed must never be compromised.

What **is** built is a storage *layer* around Postgres: copy-on-write branch drivers — ZFS on macOS/Linux, btrfs on Windows — behind one interface in `internal/branch/storage.go` (C1), quotas (C6), capacity reporting (C7), and durability to object storage with `wal-g` + MinIO: WAL archiving, base backups, point-in-time restore (D1–D3). Networked storage disaggregation is listed in the README roadmap but not started (see "Not started").

Suggested next steps

- The claims are checkable now:** a restore is proved weekly and by `fox backup verify` (D2), a tag cannot ship past a red or stale nightly (M2), every command the engine runs has a deadline so nothing hangs for ever (G30's useful half), containers restart on crash (B2), and CI runs the race detector. Before this, the most consequential thing the product claims — that the backups restore — had never been tested end to end on a real install.
- This checklist's own badge rule is enforced** (audit v2 G31): no row can be badged done while carrying an open sub-item, and `scripts/test_feature_checklist.sh` fails the build if one is. The 45 sub-items that broke the rule were each put somewhere honest — built, a limit (!), planned (→), or a note in "Tested by" — and nine of them turned out to be things we had already shipped and never ticked off, including automated UI tests, per-user access control, SDK contract tests and the first signed release.
- Run on real machines what the throwaway VM cannot: `fox update` across a real release on macOS and Windows (A9); on Windows, the `zstd` distro image's unpack (M5), the btrfs holding area of `fox pg upgrade` (D6), and an export's copy to the host (D5).
- Automated tests for the imports (I1–I3, I7) — the last area with none.
- Gate release tags on the nightly suites, so a tag cannot ship past a red suite (M2).
- Decide how this checklist shows partial work (audit v2 G31): 9 features are badged done while carrying an open sub-item (C7, D5, D6, F5, G3, H16, H17, I13, M7) — down from 34. Each of those sub-items is a disclosed limit rather than unfinished work, so the remaining question is whether a limit belongs in the sub-item list at all.
- The object store's images are FoxByte's own** (M6): MinIO left Docker Hub in 2026 and then quay.io stopped serving it anonymously — every tag there answers 401 — so a fresh install on macOS or Linux could not start, while existing installs carried on from their cache and Windows carried on because its distro image ships them inside it. They are mirrored to `ghcr.io/thefoxbyte` by `.github/workflows/mirror-images.yml`, which takes them out of a published release's distro asset rather than from a registry

that will not serve us. **Run it once and pin what it prints.** It can only be dispatched after it is on the default branch, and until the digests are pinned `release.yml` refuses to publish.

8. **Promotion is built** (H18, audit G29): a branch's recorded statements, reviewed, then applied to another branch in one transaction — attributed to whoever wrote them, with the approver beside them. Schema only. An agent can propose and cannot approve its own request.
9. **From the audit, deliberately left:** HA beyond the demo — automatic failover and fencing across hosts (D4, G21); metrics and alerts, and supervision of the background servers themselves (B2, G27 — the containers restart); splitting the branch package and running its SQL through pgx (G30 — the deadlines are in); quotas on btrfs and per-branch CPU/memory (C6, G33).

A Install & setup

7 of 9 done

ID	Feature · sub-features	Tested by
 A1	One-line installer (macOS / Linux) ✓ Detects OS / CPU, downloads the release binary ✓ Installs to the prefix (sudo only if needed) ✓ Stages the Linux engine for the VM on macOS ✓ <code>FOX_VERSION</code>, <code>FOX_REPO</code>, <code>FOX_DIST</code>, <code>FOX_PREFIX</code>	Unit (<code>test_install_sh.sh</code> , CI); manual
 A2	Windows installer (install.ps1) ✓ Installs WSL, elevates, resumes after reboot ✓ Downloads fox.exe, engine, image build context ✓ Ubuntu rootfs verified against Ubuntu's checksums ✓ Adds to PATH and runs <code>fox setup</code>	Pester (Windows CI); Windows run 16 Sep
 A3	fox setup on macOS (Lima) ✓ Creates / starts the Lima VM ✓ Installs Docker and ZFS ✓ Reinstalls the engine on every run ✓ Starts the stack	Unit; run against a deliberately broken VM, 21 Sep
 A4	fox setup on Windows (WSL2 + btrfs) ✓ Imports a dedicated distro with systemd ✓ Docker, btrfs and a storage mount unit ✓ Quiet steps, detail in install.log ✓ Waits for systemd on a cold distro	Unit (<code>host_wsl_test.go</code>); Windows run 16 Sep
 A5	Linux native ✓ Engine runs in-process, no VM ✓ <code>fox setup</code> explains nothing is needed	VM integration (all suites run on Linux)
 A6	Command forwarding & host file streaming ✓ macOS → Lima VM, starts the VM if stopped ✓ Windows → WSL distro	Unit (<code>TestImportLocalFile</code> , <code>TestWSLargs</code>)
 A7	Always-latest engine binary ✓ Downloads the newest engine into the cache (ELF check) ✓ <code>FOX_NO_REFRESH</code>, <code>FOX_VERSION</code>	Unit (<code>host_test.go</code> , <code>update_test.go</code>); Windows run 16 Sep
 A8	New-release notice on fox start ✓ Background check, 1.5 s limit, printed last ✓ Only a promoted, newer, complete release	Unit; update \$1, \$1b, \$2, \$3, \$8; real Mac check
 A9	fox update ✓ Downloads and verifies everything before changing anything ✓ Stops servers, installs engine, restarts old one on failure ✓ up, Blackbox upgrade, <code>start</code> on the new engine ✓ Replaces host <code>fox</code>, keeps previous binaries ✓ <code>--check</code>, <code>--yes</code>, <code>--version</code> ✓ Linux path verified end to end ✓ macOS path run against real releases (v0.8.3 → v0.8.4 → v0.8.5)	Unit; update \$4–\$7; a real Mac run

B CLI & services

3 of 3 done

ID	Feature · sub-features	Tested by
 B1	The fox command set ✓ Stack: setup, start, stop, up, down, status, check, logs, connect, psql, demo, update ✓ Branching, backup / restore, HA ✓ Blackbox, policy, impact ✓ Import, pipelines ✓ backup export / restore, pg status / upgrade	Unit (<code>main_test.go</code> , <code>update_test.go</code>); VM suites
 B2	Background services (daemon) ✓ Control plane :8080, gateway :6432, agent API :8088 ✓ Detached, pidfiles and logs in <code>~/ .fox</code> ✓ Stop (TERM then KILL), status, logs	Unit (<code>TestStopTerminatesProcess</code>); VM \$1 ! The three background servers themselves are not supervised (<code>fox check</code> reports them)

✓	B3 First run (sample data, check, connect)	Unit; v1 §11d
✓	- Sample data seeded into <code>main</code> on a first start: <code>users</code>, <code>projects</code>, <code>events</code>, five rows each, two foreign keys <code>fox demo seed drop sql</code> on any branch; <code>FOX_NO_DEMO=1</code> starts empty - The console opens itself, with the one-time setup token already in the link - Declines to open a browser in the VM, over SSH, with no display, or with <code>FOX_NO_BROWSER</code>	<code>fox check</code>: version, console, storage, main, three servers, three ports, TLS, Blackbox, backup age, sample data — each failure with the command that fixes it <code>fox connect [branch]</code> prints the application's connection string, then opens <code>psql</code>; <code>fox psql</code> takes a branch <code>fox branch create</code> ends with the connection string and how long it took - Plain-English errors in the CLI and the console (<code>internal/errhint</code>, <code>web/src/errhint.ts</code>)

C Branching & storage

7 of 8 done

ID	Feature · sub-features	Tested by
✓	C1 Storage drivers (ZFS / btrfs) - ZFS snapshot + clone per branch (macOS VM, Linux) - btrfs subvolume snapshots (Windows) - Pool / filesystem created on a loopback file	Unit; VM §2, §2b (ZFS); btrfs manual on Windows
✓	C2 Branch lifecycle - Create, list, delete - Reset to parent - Suspend / resume - Recreate a container from its surviving data	Unit; VM §2, §2b, §4
✓	C3 Protection of <code>main</code> Delete and reset refused	Unit; VM §4, §6
✓	C4 Private networking (no host ports) - Branch databases reachable only through the gateway - Published ports (object store, restore, debug) on loopback only; standby unpublished	Unit (<code>TestParsePublishedPort</code> , <code>TestDSN</code>); VM §1c
✓	C5 Scale-to-zero & wake on connect - Idle branches suspended automatically - Next connection wakes the branch (~0.3–0.7 s)	VM §7
🔥	C6 Storage quotas - Space reserved for main (ZFS) - Per-branch cap (opt-in)	Manual
✓	C7 Capacity & status - Services, main health, backups, branches - Storage used / available	VM §1
✓	C8 Provisioning & image pull - Docker, storage pool and image prepared on <code>up</code> - Pulls from GHCR, builds locally as fallback	Unit; VM §11b

D Durability

5 of 6 done

ID	Feature · sub-features	Tested by
✓	D1 WAL archiving to object storage - wal-g ships main's WAL to MinIO, off the commit path - MinIO images pinned on quay.io - Branches are disposable by design (export one that matters)	VM §3; VM §11c; not yet run against AWS S3
✓	D2 Base backups - Create and list - Follow the primary after a failover; listing works with main stopped - Read-only listing: <code>GET /api/backups</code> and a Dashboard panel - Schedule and retention (<code>FOX_BACKUP_INTERVAL</code>, <code>FOX_BACKUP_RETAIN</code>) - A first backup at install	VM §3, §3b, §6; VM §11c
✓	D3 Point-in-time restore - Restore to a timestamp or latest on :5433 - Per-install credentials	Unit; VM §3, §3b, §6; v2 §1a
🔥	D4 HA standby & failover - Streaming hot standby - Manual failover, <code>main</code> rerouted - Nothing deletes or stops the promoted standby <code>fox up</code> brings it back, not the old main	Unit; VM §6, §7

✓ D5	Portable export & restore ✓ Every non-agent branch: pg_dump, its roles, the Blackbox head; SHA256SUMS over all ✓ Copied out of the VM and verified on the host; a copy that fails is deleted	✓ Damaged, incomplete or path-escaping archives refused ✓ Restore into a new branch, same or newer major; Blackbox checked exact	Unit; VM pg-upgrade §1; the host copy on macOS/Windows is unit-tested only
✓ D6	PostgreSQL major upgrade ✓ fox pg_status : the major of the install and of each branch ✓ Plan with every change crossed, found-in-your-data first; refusals say what to do ✓ Asks, exports, then reloads every carried branch on the new major	✓ Data, roles, API keys and every Blackbox entry come across ✓ Old databases held outside the branch namespace; rollback and finalize ✓ Each upgraded cluster archives WAL under its own prefix → Carry agent branches; upgrade with HA on	Unit; VM pg-upgrade (72 / 72); the btrfs (Windows) holding area has not been run

E Security foundations

3 of 3 done

ID	Feature · sub-features	Tested by
✓ E1	Per-install generated secrets ✓ Generated Postgres and MinIO passwords ✓ Used by restore too	Unit (indirect); v2 §1a; VM §1d
✓ E2	TLS everywhere ✓ Self-signed TLS on all three services, zero setup ✓ TLS required from other machines (loopback clients may skip it)	Unit (TestReadStartupUpgradesTLS)
✓ E3	Keys are made by a person, not by start ✓ Start creates no account and no key, and caches none on disk ✓ The banner shows the connection string and how to make a key	Integration §1

F SQL gateway

7 of 7 done

ID	Feature · sub-features	Tested by
✓ F1	Routing by database name ✓ One endpoint :6432, database name = branch	Unit; VM §1, §6
✓ F2	API-key authentication ✓ key_ key as the Postgres password ✓ Bad key rejected with a Postgres error	VM §1
✓ F3	Backend login (SCRAM) ✓ Cleartext, md5, SCRAM-SHA-256	VM §1, §6, §8 (exercised); no unit test for SCRAM yet
✓ F4	Per-user Postgres login roles ✓ Role per user, actor can't be forged ✓ Permissions of non-superuser db_client	Manual
✓ F5	Attribution session settings ✓ Actor kind, branch, session id per connection	VM §8
✓ F6	Ctrl-C (CancelRequest) forwarding ✓ Cancel a running query through the gateway	Unit
✓ F7	Errors & insecure build tag ✓ Real Postgres error responses	Unit (TestSendError)

G Accounts & auth

5 of 6 done

ID	Feature · sub-features	Tested by
✓ G1	Auth store (SQLite) ✓ Users, keys, sessions, OAuth ids, pipelines ✓ Safe under concurrent use (WAL, busy timeout), including a new store created by three services at once	Unit (TestVerifyKeyConcurrent , TestConcurrentFirstOpen); VM §1
✓ G2	Email & password users ✓ Create account, log in ✓ Password change (signs out other sessions); admin reset at the CLI → Self-service reset by email ✓ List and delete users (admins)	Unit; VM §1; VM §1d
✓ G3	API keys ✓ Create (shown once), list, revoke ✓ Bearer / X-API-Key and gateway password	Unit; VM §1
✓ G4	Web sessions & signup control ✓ httpOnly session cookie ✓ Sign-up closed by default (FOX_SIGNUP=open)	Unit (TestSessions)

	G5 OAuth (GitHub / Google) ✓ Sign-in flows built ✓ Defaults match what the engine serves (<code>https://localhost:8080</code> ; web origin = public URL) ✓ Session cookie <code>Secure</code> / <code>SameSite</code> follow the origin	□ Tested with a real OAuth app ✓ Only a provider-verified email joins or creates an account; closed sign-up respected ✓ State cookie one-use, <code>Secure</code> , <code>SameSite=Lax</code>	Unit; VM §1b
	G6 Security log ✓ Sign-ins, failures, keys, passwords, accounts, admin grants, refusals ✓ Hash-chained, append-only, anchored and signed	✓ <code>fox audit</code> , <code>verify</code> , <code>GET /api/audit</code> → Included in <code>fox backup export</code>	Unit; VM §1d

H

Blackbox (record of schema changes)

16 of 18 done

ID	Feature · sub-features	Tested by
 H1	DDL recording ✓ Who, tool, branch, statement, status, risk ✓ Inherited by every branch ✓ A restart records nothing: the install script's own statements are not captured ! Role / database changes, some dropped object types ✓ Agents' UPDATE and DELETE recorded; TRUNCATE by anyone	✓ An agent's UPDATE or DELETE with no WHERE is refused, and the attempt recorded — a superuser or db_admin can let one through ✓ Triggers on every table, protected from drop/disable/rename VM §8; v2 §9; v2 §10
 H2	Hash chain, append-only & verify ✓ Rows can't be edited or deleted normally ✓ fox ledger verify  Superuser rewrite caught only via anchors (H10–H12)	✓ bb.policy changes recorded (append-only, not chained) VM §8; v2 §0, §6b
 H3	Attribution ✓ Humans via their login role ✓ Agents via their own agent-<id> role	✓ Kind derived from the role, so neither can pose as the other ✓ Tool and session are client-declared by design, and labelled so Unit; VM §8; v2 §1c
 H4	Destructive-DDL guardrail ✓ DROP TABLE / DROP SCHEMA refused, attempt recorded ✓ Override only superusers / db_admin, use recorded	✓ "flag" and "allow" actions; unknown actions refused ✓ No hang when blocked after a write in the same transaction ✓ TRUNCATE blocked by default, recorded VM §8; v2 §1b, §6b
 H5	Risk flags ✓ Type change, drop column flagged ✓ Judged on the running statement (no comments or literals)	✓ Automated test Unit; v2 §6b
 H6	Blackbox access (CLI / REST / MCP / UI / SDK) ✓ CLI, REST with filters, MCP, web page, SDKs	✓ Per-user access control (internal/access: owner or admin) v2 §0, §5
 H7	fox ledger revert --to ✓ Help and output say it's a point-in-time restore of main	✓ Points to H13 for a branch from before a change Unit (help text); VM §3 (the restore)
 H8	Override permission (db_admin) & upgrade ✓ fox admin grant revoke list ✓ First user granted automatically	✓ fox blackbox upgrade [--all] ✓ Grant / revoke from the web (admins only) v2 §1b, §6
 H9	Blackbox 2.0 capture ✓ Transaction id and WAL position per entry ✓ Task, tool-call hash, override use	✓ Fail-safe, append-only ✓ Kill switch not settable by clients Unit; v2 §2
 H10	Checkpoints & anchors ✓ Merkle root in the database and in read-only anchor files ✓ On a schedule and on demand ✓ Published anchor format	✓ Copied to the backup target (S3 Object Lock) ✓ Anchors signed (Ed25519, key outside the databases) Unit; v2 §3, §3b
 H11	fox-verify (independent verifier) ✓ Checks a live branch or an export ✓ Release binaries for every platform	✓ --pubkey checks signatures Unit; v2 §3
 H12	Blackbox integrity & export ✓ INTACT / TAMPERED via CLI, REST, MCP ✓ JSON-lines export	→ Streaming for very large ledgers v2 §3
 H13	Branch from just before a Blackbox entry ✓ Restore + replay up to the change's transaction ✓ By transaction id or timestamp ✓ CLI, REST, MCP, the Integrity panel, and every row of the Blackbox page	✓ main untouched ✓ Documented: README, the Guide's step 6, and the reference Unit; v2 §4; VM ui
 H14	Blackbox names & aliases ✓ fox blackbox, /blackbox routes, MCP and SDK names	✓ Every original name still works Unit; v2 §5
 H15	Policy gate (warn / block) ✓ BBX02 warn / BBX01 block with JSON detail ✓ Blocked attempts recorded ✓ Per-rule db_admin override	✓ CLI, REST, MCP, web page; custom rules ✓ Patterns match the running statement ✓ TRUNCATE (its own guardrail, H4) Unit; v2 §6, §6b

✓ H16	Agent provenance & execute_change ✓ Session, task, parent session on every agent change (also through the Gateway DSN) ✓ MCP execute_change with policy preview	✓ fox_blackbox_sessions, REST ! Provenance is what the agent says (not verified)	Unit; v2 §7
🔗 H17	Impact analysis ✓ Dependents (views, foreign keys, indexes, depth) ✓ Other branches that have the object ✓ Policy verdict and risk score	✓ CLI, REST, MCP, inside execute_change 🔗 Functions, types, schemas not recognised from SQL	Unit; v2 §8
✓ H18	Blackbox diff, and promoting a branch's changes ✓ Fork point, changes only on each side ✓ Objects changed on both ✓ Diff: CLI, REST, MCP ✓ Promotion: a branch's recorded statements, reviewed, then applied to another branch in one transaction	✓ Refused on a conflict (both sides changed the same object), on a statement the target's gate blocks, and when there is nothing to promote ✓ Approving needs the right to manage the target, so an agent cannot approve its own request ✓ Promotion: CLI, REST, MCP and the web Requests page	Unit; v2 §8, §8b

I Import & migration

13 of 13 done

ID	Feature · sub-features	Tested by
✓ I1	From Postgres ✓ Schema and data via pg_dump	Manual; no automated test yet
✓ I2	From MySQL ≤5.7 / MariaDB ✓ pgloader with type mapping (multi-arch image)	Manual; no automated test yet
✓ I3	From MySQL 8.x ✓ Native mysqldump path (caching_sha2 auth) ✓ Types, NULLs, unicode, PK	Manual; no automated test yet
✓ I4	From MongoDB ✓ Collection → typed table, exact name	VM §9, §10
✓ I5	From a .sql dump ✓ Runs the dump into a new branch ✓ Fails listing every failed statement (instance kept)	Unit; VM §11
✓ I6	From JSON / NDJSON ✓ Columns inferred from top-level keys	Unit; VM §10, §11
✓ I7	From CSV ✓ Table from the header, exact column names	Manual
✓ I8	Continuous replication & cutover ✓ Initial copy + streaming (logical replication) ✓ Cutover keeps data (CLI) ✓ Status and cutover from REST and the Import page	Unit; VM §5b
✓ I9	Guard handling during import ✓ Guardrail paused, recording stays on ✓ Re-armed even after a crash	Manual; integration_test.sh §5b
✓ I10	Live progress (CLI & SSE) ✓ Live log and progress bar	Manual
✓ I11	Import REST endpoints ✓ From a connection string or an uploaded file ✓ Target names validated (the import creates the branch through branch.Create)	Manual; unit; VM §1d
✓ I12	Web Import page ✓ Database and file panels with live console	Manual
✓ I13	Exact identifier fidelity ✓ Case kept (userId, leaseAiChats)	Unit; VM §10

J ETL pipelines (experimental)

4 of 4 done

ID	Feature · sub-features	Tested by
✓ J1	Pipeline engine ✓ Extract → raw schema → SQL models → tests, in a fresh branch ✓ ref() / source() templating → Dependency graph	Unit; VM §9
✓ J2	Data-quality tests ✓ not_null, unique, accepted_values, row_count_min, custom	Unit; VM §9

✓	J3 Stored pipelines & run history ✓ Saved per user, run over the API, history → Read a run's log	→ Mark runs whose process died	Manual
✓	J4 Pipeline editor (web) ✓ Create, edit, run, results, history	→ Confirm before delete	Manual

K APIs, SDKs & agents

6 of 6 done

ID	Feature · sub-features	Tested by
✓ K1 Control-plane REST API ✓ Branches, query, Blackbox, policy, impact, import, pipelines, keys ✓ Versioned paths: <code>/api/v1/...</code> reaches the same routes, and <code>/api/...</code> keeps working	✓ Exact-origin CORS, branch-name check, body limits, timeouts ✓ Branches owned by their maker; others 404 (owner, admin, or main)	Unit; VM \$1; v2 \$0–\$8; VM \$1d
✓ K2 OpenAPI spec ✓ Served, Blackbox 2.0 endpoints documented ✓ Matches the code (import fields, status codes, auth routes, /blackbox aliases)	✓ Drift check in CI	Unit (drift check)
✓ K3 Python SDK ✓ Branches, query, Blackbox → Import, pipelines, keys, agents ✓ Contract tests against a live engine (nightly sdks suite)	→ Publish to PyPI ✓ Contract tests against a live engine	Manual; VM sdks
✓ K4 TypeScript SDK ✓ Same client for Node and browsers ✓ Contract tests against a live engine (nightly sdks suite)	→ Publish to npm ✓ Compiles, and contract tests against a live engine	Manual; VM sdks
✓ K5 Agent Branch API ✓ A branch per agent, own non-superuser role ✓ Provenance (task, parent, session) ✓ DSN reachable from the host (gateway-routed, branch-scoped key)	✓ Cap, listing and TTL count suspended branches ✓ Superuser switch only in <code>-tags insecure</code> builds ✓ Agents and their keys belong to the caller	Unit; VM \$5; v2 \$1c, \$7
✓ K6 MCP server ✓ Branches, run_sql, Blackbox tools ✓ <code>policy_check</code> , <code>execute_change</code> , <code>impact</code> , <code>diff</code> ✓ Requires an API key and acts as that account; a scoped key is held to its branch	✓ Client config & 16-tool reference (docs/mcp.md) ✓ Same ownership rule as the REST API	Unit; v2 \$1c–\$8

L Web console

11 of 11 done

ID	Feature · sub-features	Tested by
✓ L1 Embedded single-origin UI ✓ Served by the engine at :8080 ✓ Automated UI tests (Playwright, in the nightly ui suite)	✓ One branch-list loader for every selector: retries a dropped request, reports one it cannot load	Manual
✓ L2 Login & signup ✓ Log in / sign up tabs, OAuth buttons	✓ Covered by the console's Playwright tests	Manual; VM ui
✓ L3 Dashboard ✓ Live tiles ✓ Branch table: create, suspend, resume, delete, copy DSN	✓ Covered by the console's Playwright tests	Manual; VM ui
✓ L4 SQL console & browser ✓ Schema sidebar, rows, structure, indexes ✓ Values shown as Postgres writes them (uuid, interval, inet, bytea) ✓ SQL editor ✓ System tables hidden by default, with a show button ✓ Runs as the signed-in user; one-run override switch ✓ Per-rule override for policy blocks ! One statement per run (extended protocol) ✓ Runs a script, not just one statement: a result per statement, and one transaction unless it opens its own ✓ Each result says which statement produced it; labelling is dropped rather than guessed when a split is unsafe	✓ Editor is read-only while a query runs, and <code>⌘/Ctrl+↵</code> no longer starts a second one ✓ Workbench layout: fills the window, schema rail folds away, results in tabs, either pane expands ✓ Line numbers, and a syntax error reports its line and column and marks the gutter ✓ Component tests for the console run on every pull request, without a stack ✓ Page heading kept for screen readers after the title block became a toolbar	Manual; API in v2 \$6
✓ L5 Blackbox page ✓ Entries per branch with badges ✓ Table / risk / date filters, and "Clear filters" ✓ Verify button: a row expands to its statement and session ✓ Live: new entries arrive on their own (3s), marked briefly, pausable; never while older pages are being read	✓ The chain is visible: entry id, its row hash and the previous entry's, each copyable ✓ "Branch from before this change" on the row itself (<code>main</code> ; explained elsewhere) ✓ Covered by the console's Playwright tests	v2 \$5b; browser; VM ui
✓ L6 API keys page ✓ Create, copy, list, revoke ✓ A new key is shown inside its connection string (E3's next step)	✓ Covered by the console's Playwright tests	Manual; browser review 21 Sep; VM ui

✓ L7	Landing, Guide & Docs pages ✓ Landing, developer guide, reference	✓ Refreshed against the code (https, sslmode, MCP, Blackbox, import routes)	Type-check + build
✓ L8	Blackbox integrity page ✓ Verify, checkpoint, export, branch-before ✓ Used signed in; every result matched the CLI	✓ New entries arrive on their own (3s), marked briefly, pausable	Type-check + build; browser review 16 Sep
✓ L9	Policies page ✓ Rules, warn/block, custom rules, preview, evaluations ✓ Used signed in; matched the CLI, wrote only to the chosen branch	✓ Non-admin change refused with a readable message ✓ Who can override: list, grant, revoke	Type-check + build; API in v2 §6; browser review 16 Sep
✓ L10	App sidebar layout ✓ Every link opens its page, highlighted, with breadcrumb ✓ Collapse persists across a reload; theme toggle persists	✓ No console errors on any page	Type-check + build; browser review 16 Sep; the under-860px drawer needs a manual look
✓ L11	Design system & FoxByte theme ✓ Tokens for colour, type, space, radius, elevation and motion; components use no raw values ✓ Ember palette in both themes; filled controls legible (<code>--accent-ink</code>)	✓ Logo artwork as the mark, wordmark from brand.json accenting the logo's letter, favicon and touch icon ✓ Console, Blackbox, Policies and API keys re-laid out; one table style ✓ Every page reviewed in both themes; console checked at 375px	Type-check + build; browser review 21 Sep; not yet reviewed against real data

M Build, CI & releases

8 of 9 done

Policy authoring, impact, promotion, export and pipelines gated; reading each stays free

All five moved: `enterprise/{pipeline,impact,promote,anchor,policy}` — ~950 lines, and zero enterprise symbols in the Standard binary

SQL schema split: `ledger_ext`, `provenance` and `datachanges` stay free; `checkpoints`, `policy` and `impact` moved to `enterprise/schema` and are installed per feature

Realtime foundation: opt-in, container settings (byte-identical when off), slot naming and sweeping, the suspend probe, preflight refusals

Realtime decoder: wire format, fan-out, pgoutput decoding — all in `enterprise/realtime`

SSE route on the outer mux, branch-scoped key (`UserForBranchStream`), decoder lifecycle, `realtime enable|disable|tables`, console page

Realtime integration suite (`make integration-realtime`, in the nightly matrix): refusals, the wire format against a real WAL, scoped keys, slot sweeping, teardown, replay via `?since=`, and the WAL budget invalidating a slot

ID	Feature · sub-features	Tested by
✓ M1	Makefile targets ✓ build, vet, test, integration suites ✓ Suites run only in a throwaway test VM (guarded)	CI
✓ M2	Linux CI ✓ Build, vet, test, gofmt ✓ Release script tests, checklist sync check ✓ VM integration suites nightly in CI (one runner per suite)	CI
✓ M3	Windows CI ✓ Build, vet, test, Pester, installer encoding	CI; no end-to-end WSL install test yet
✓ M4	Releases & GHCR image ✓ Binaries for 5 platforms + fox-verify ✓ SHA256SUMS merged safely between jobs ✓ Fails unless <code>fox update</code> 's files are published	Script tests in CI
✓ M5	WSL distro & ZFS builds ✓ Prebuilt distro image ✓ Downloaded and verified by the installer (seen on Windows) ✓ Preloaded image tagged as the engine runs it	Actions artifacts; Pester; Windows run 16 Sep; the <code>zstd</code> unpack is unit-tested only, not run on a real Windows install
✓ M6	Postgres image ✓ PostgreSQL 18 + wal-g for a fresh install; one image per supported major (16, 18) ✓ The image follows the data: an install on 16 keeps running 16 after an update ✓ A branch on the wrong major is refused before <code>docker run</code> , with the way out ✓ The major lives in one place, and a test holds the Dockerfile, release matrix and Windows preload to it	Unit; all three suites on 18
✓ M7	Product name in one file ✓ <code>brand.json</code> → Go, shell, web and installer blocks, regenerated by <code>make brand</code> ✓ Stored names brand-free and frozen: a rename never migrates an install	Unit (brand, gen, oldnames), in CI via go test

✓ M8	Two editions		Unit (edition, boundary), scripts/test_editions.sh, CI builds both
	✓ One tree, two binaries: <code>fox</code> (Standard, AGPL) and <code>fox-enterprise</code> ✓ <code>enterprise/</code> has its own commercial LICENSE and LicenseRef- headers ✓ The Standard binary contains no paid code — checked with <code>go tool nm</code>, not just the source ✓ An Enterprise build with no licence behaves exactly like Standard	✓ Installer <code>--edition / FOX_EDITION</code>; launcher and VM engine always match ✓ <code>fox update</code> stays inside its edition; the release publishes and checks both sets ! Source-available deters, it does not prevent: the check can be removed in breach ! A dependency only the paid code uses still appears in the AGPL module's <code>go.mod</code>	
🔗 M9	Licensing		internal/license
	✓ Activating from the console: <code>POST /api/license</code> and an Account → Licence page — admin only, a forgery refused outright, honoured by this process at once, with the restart and the VM-copy caveats stated rather than hidden ✓ Ed25519 over a canonical payload, with a key id so the key can rotate ✓ <code>cmd/licensesign</code>: generate, issue, verify; private key never in CI ✓ Soft machine lock, grace before refusal, and a lapse that never stops a database ✓ Host fingerprint: hashed, read on the host not in the VM, failure means unknown	✓ <code>~/fox/license.json</code> (0600, atomic) and <code>fox license activate show rebind remove</code> ✓ The engine inside the VM is given a copy (macOS/Windows); no-op on Linux ✓ Security-log records: <code>license.{activated,rebound,lapsed,removed}</code>, written by the engine at start-up ✓ <code>GET /api/license</code> (state for everyone, customer and binding for admins) ✓ The console's notice: silent when active, silent in Standard, dismissible only while it still unlocks ✓ Anchors gated: writing is Enterprise, checking (<code>integrity</code>, <code>fox-verify</code>, <code>anchor-key</code>) and the security log's anchors stay free	

N Realtime change feed

1 of 1 done

ID	Item · scope	Tested by
✓ N1	Realtime change feed	enterprise/realtime, scripts/integration_realtime.sh
	✓ Readiness as a verdict, not a refusal: <code>realtime doctor</code> classifies every table, <code>realtime prepare</code> makes the changes and prints them first ✓ The cost ladder — primary key, then a unique index as the identity (same price, no schema change), then inserts-only, and <code>REPLICA IDENTITY FULL</code> last; <code>--all</code> never applies a costly fix ✓ Only application tables stream: not <code>bb</code>, not Postgres's, not an extension's — and the application's own schemas now do ✓ Opt-in: <code>fox realtime setup --yes</code> says what it will restart; with the feed off, a primary's arguments are byte-identical to before and a branch gets none ✓ Durable replay through <code>?since=<commit_lsn></code>, bounded by <code>max_slot_wal_keep_size</code>; past it the subscriber is told to <code>resync</code> rather than left waiting ✓ One decoder per branch in the control plane, started on the first subscriber and stopped after the last; it wakes a suspended branch ✓ Refusals that hold: row-level security (no override), no primary key (and <code>UPDATE</code> still works afterwards), anything outside <code>public</code>, the Blackbox's own tables, anything <code>db_client</code> cannot read ✓ Wire format: values as text, identity always, <code>old</code> only when sent, <code>changed</code> only under <code>FULL</code>, unchanged-TOAST named rather than nulled ✓ Both answers to "what was the row that is gone", in one capture against a live WAL: a delete under the default identity carries the key alone, and under <code>FULL</code> the whole old row — where every column is part of the identity too ✓ SSE on the outer mux so a branch-scoped key reaches its own branch and nothing else; somebody else's branch is 404, not 403 ✓ Slots named, listed, accounted for and swept only once Postgres has marked them lost; <code>fox check</code> warns on one nothing is draining ✓ The reaper leaves a subscribed branch alone (asked of Postgres, not of memory), and a resumed branch gets a fresh idle window ✓ A failover keeps the feed: the standby carries the same arguments, <code>setup</code> restarts one that predates it, and the feed follows <code>main</code> onto the promoted standby ✓ Neighbours unaffected: a continuous import runs under logical decoding and is itself decodable; backups, verify, failover and fallback all pass ✓ <code>setup</code>, <code>teardown</code>, <code>status</code> and <code>slots</code> stay in the free build, so a downgrade can still undo it ! Console and REST only this release; the published SDKs get the feed once the wire format has survived real use ! No per-subscriber row filters, which is why an RLS table is refused rather than filtered ✓ Published clients subscribe: <code>clients/typescript</code> and <code>clients/python</code>, dependency-free, with whole-transaction delivery, resume only at a commit boundary, reconnection, and an <code>apply</code> that leaves an unchanged TOASTed column alone	✓ 41 client unit tests (18 TypeScript via <code>node --test</code>, 23 Python via <code>unittest</code>) plus contract tests driving both against a live engine with a real <code>rtk_key</code> ✓ Transaction framing (<code>?transactions=1</code>): <code>begin / commit</code> around each transaction, every change carrying its <code>xid</code> and the transaction's <code>commit_lsn</code> as the resume boundary, with the record's own position as <code>lsn</code> ✓ Off by default and dropped per subscriber, so an unframed client is unaffected; an empty transaction produces no frame; a value other than 1 or 0 is refused rather than assumed ✓ What staying warm costs: <code>realtime activity</code>, an API route and a console panel — warm-since from the container, work from <code>pg_stat_database</code> sampled on a timer, and the bookmarks' held WAL against the budget ✓ Transactions rather than queries, because that is what Postgres counts without <code>pg_stat_statements</code>; and five cases where subtracting two readings would lie are each answered with a note instead of a figure ✓ A cap of 8 subscribers per branch (<code>FOX_REALTIME_MAX_SUBSCRIBERS</code>), refused with the count, so a client reconnecting on a bug cannot add streams until something runs out ✓ Console: Tables, Connecting an application, and Feed — verdicts become buttons, and the one fix with an ongoing cost asks first with the measured figure in the dialog ✓ Console routes on the inner <code>/api/ mux (enable, disable, prepare, realtime keys)</code>, all <code>Manage</code> by prefix; reading verdicts reuses the front door so the console cannot drift from what an application sees ✓ Covered in a browser: <code>web/tests/realtime.spec.ts</code> classifies a real branch, applies a free fix, declines a costly one, publishes and unpublishes a table, and mints then revokes a key ✓ Front door: <code>/realtime/v1/branches/{name}/{hello,tables,stream}</code> on the console's port, and a <code>fox-realtime:// connection</code> string that keeps the key in <code>userinfo</code>, never in a URL ✓ A <code>realtime key kind (rtk_, fox realtime key create ls revoke)</code>: streams one branch, refused at the control plane and refused at the Gateway by kind — asserted against a real <code>psql</code> connection ✓ Only an application's tables over HTTP: one shared <code>realtime.Application()</code> withholds <code>bb</code>, Postgres and extension-owned tables and reports how many — the first version served the Blackbox's shape to a stream-only key ✓ Additive: the original <code>/api/branches/{name}/realtime</code> route is unchanged, and every key issued before kinds existed keeps the empty kind and its old behaviour (migration covered from a pre-kind store) ! A delete under the default replica identity carries the primary key and nothing else

Planned or discussed, nothing built yet. When work starts, move the row into its area with the next id there.

ID	Item · scope	Related
☐	Multi-tenant projects ☐ Projects / organisations, and roles within them ☒ Per-branch permissions: owner or admin, on every branch-scoped route (G2 of the audit)	G1, H6
☐	Automatic HA failover ☐ Health detection, fencing ☐ Multi-host	D4
☐	Pipelines: scheduling, incremental / CDC extract, DAG view	J1
☐	More import sources: Cassandra, DynamoDB	I1–I8
☒	Release signatures — done 22 Sep 2026	A9, M4
☐	Hosted cloud tier	—
☐	Networked storage disaggregation (README roadmap; see the storage engine note on page 1)	C1, D1

Out of scope by decision

A custom storage engine (PostgreSQL's own engine is used, for native transaction speed), sharding, Raft / cross-node transactions, multi-region, HTAP.